Metro Ticket Generating System

Phase 2-Backend Development and Configurations

Data Architecture

Data architecture defines how ticket data is stored, organized, and managed within the ServiceNow platform. In the Metro Ticket Generating System, a custom database structure was designed to store ticket information, automate record creation, and support QR code generation and ticket tracking.

The backend database structure was implemented using a custom table called Metro Ticket along with multiple fields that store travel and ticket information. This data architecture ensures that all ticket bookings are properly recorded and can be managed efficiently by the system.

Subtask 1: Creation of Table

A custom table named Metro Ticket (u_metro_ticket) was created in ServiceNow to store all ticket-related information generated during the booking process.

This table acts as the primary data storage component for the metro ticketing system. Each ticket booking submitted through the Service Catalog automatically creates a record in this table through Flow Designer automation.

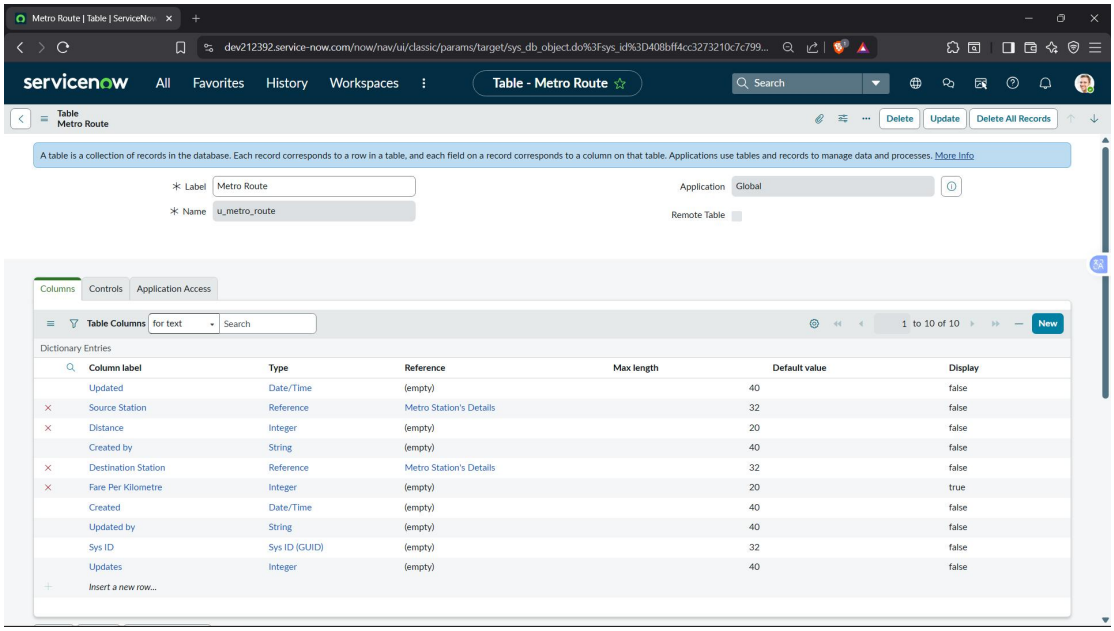
The table is responsible for storing key ticket information such as passenger details, travel details, fare amount, ticket status, and QR code.

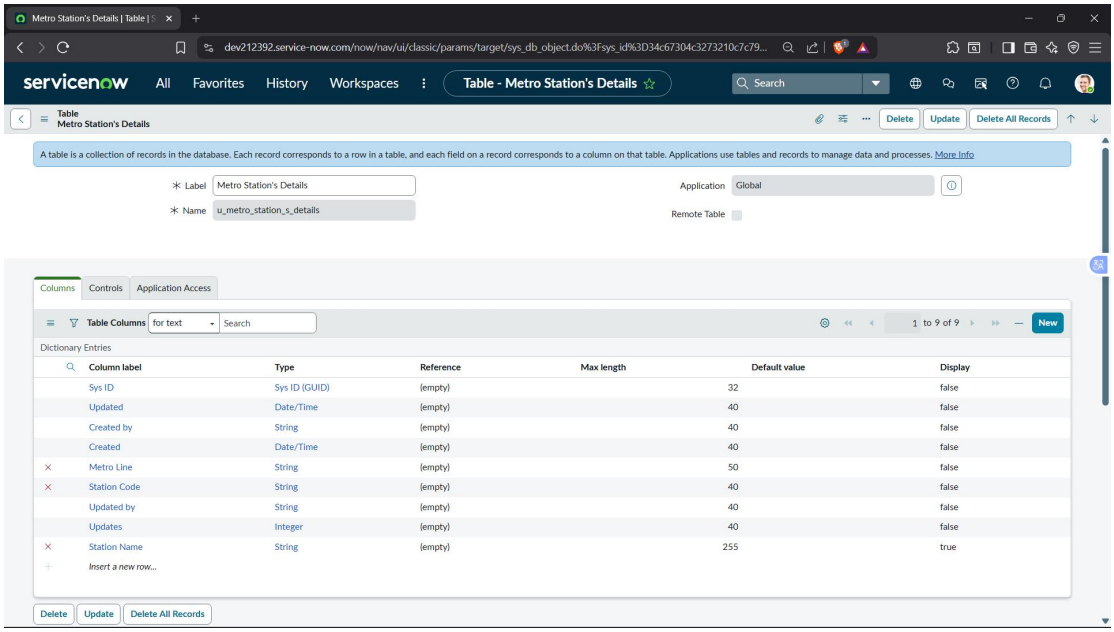
Key Purpose of the Table

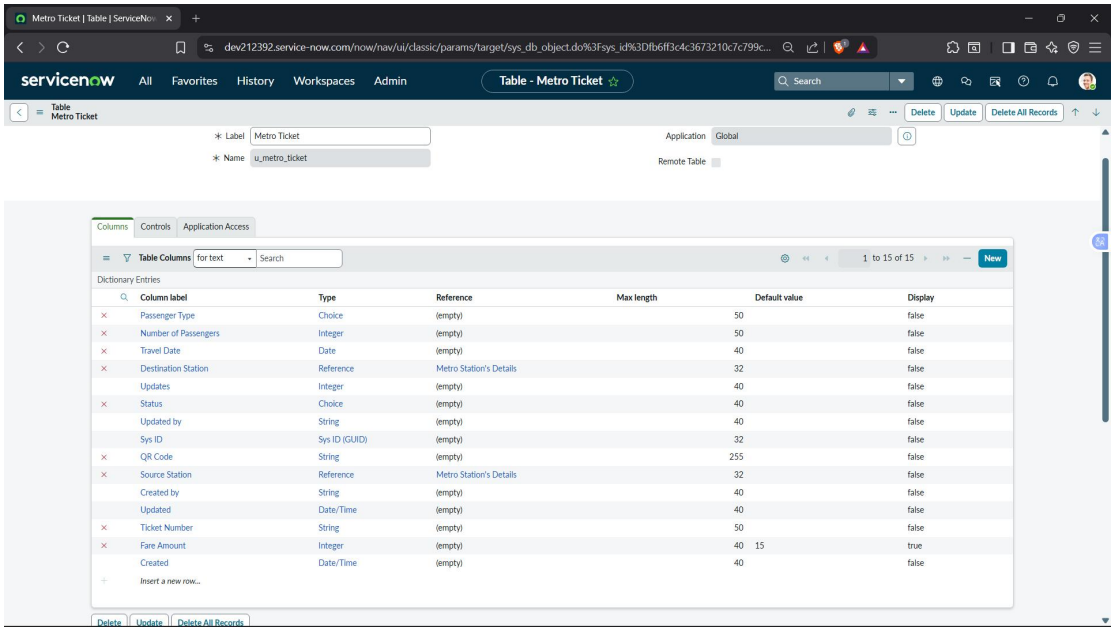
Store all generated metro ticket records
Maintain ticket history
Enable tracking of passenger travel details
Support automated workflows and notifications
Provide data for QR code generation

Table Name

Metro Ticket (u_metro_ticket)


Metro Route | Table | ServiceNow
dev212392.service-now.com/now/nav/ui/classic/params/target/sys_db_object.do%3Fsys_id%3D408bff4cc3273210c7c799...
servicenow
All
Favorites
History
Workspaces
Table - Metro Route
Search
Table
Metro Route
Delete
Update
Delete All Records
A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes. More Info
Label
Metro Route
Name
u_metro_route
Application
Global
Remote Table
Columns
Controls
Application Access
Table Columns
for text
Search
1 to 10 of 10
New
Dictionary Entries
Column label
Type
Reference
Max length
Default value
Display
Updated
Date/Time
(empty)
40
false
Source Station
Reference
Metro Station's Details
32
false
Distance
Integer
(empty)
20
false
Created by
String
(empty)
40
false
Destination Station
Reference
Metro Station's Details
32
false
Fare Per Kilometre
Integer
(empty)
20
true
Created
Date/Time
(empty)
40
false
Updated by
String
(empty)
40
false
Sys ID
Sys ID (GUID)
(empty)
32
false
Updates
Integer
(empty)
40
false
Insert a new row...


Metro Station's Details | Table |
dev212392.service-now.com/now/nav/ui/classic/params/target/sys_db_object.do%3Fsys_id%3D34c67304c3273210c7c79...
servicenow
All
Favorites
History
Workspaces
Table - Metro Station's Details
Search
Table
Metro Station's Details
Delete
Update
Delete All Records
A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes. More Info
Label
Metro Station's Details
Name
u_metro_station_s_details
Application
Global
Remote Table
Columns
Controls
Application Access
Table Columns
for text
Search
1 to 9 of 9
New
Dictionary Entries
Column label
Type
Reference
Max length
Default value
Display
Sys ID
Sys ID (GUID)
(empty)
32
false
Updated
Date/Time
(empty)
40
false
Created by
String
(empty)
40
false
Created
Date/Time
(empty)
40
false
Metro Line
String
(empty)
50
false
Station Code
String
(empty)
40
false
Updated by
String
(empty)
40
false
Updates
Integer
(empty)
40
false
Station Name
String
(empty)
255
true
Insert a new row...
Delete
Update
Delete All Records



Subtask 2: Creation of Fields

After creating the table, multiple fields were added to store all required ticket information. These fields represent the attributes of each metro ticket record.

The fields were designed to capture both user input data and system-generated information.

Important Fields Created

Field Name	Type	Purpose
Ticket Number	String	Unique identifier for each ticket
Source Station	Reference	Starting station of the journey
Destination Station	Reference	Ending station of the journey
Passenger Type	Choice	Type of passenger (Adult / Child / Senior Citizen)
Number of Passengers	Integer	Total passengers for the booking
Travel Date	Date	Date of travel
Fare Amount	Integer	Calculated fare for the journey
Status	Choice	Ticket status (Generated, Pending, Approved)
QR Code	String	URL used to generate QR code

These fields ensure that every ticket contains complete travel information and can be used for validation and system tracking.

Columns | Controls | Application Access

Table Columns for text Search 1 to 15 of 15 New

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Passenger Type	Choice	(empty)	50		false
Number of Passengers	Integer	(empty)	50		false
Travel Date	Date	(empty)	40		false
Destination Station	Reference	Metro Station's Details	32		false
Updates	Integer	(empty)	40		false
Status	Choice	(empty)	40		false
Updated by	String	(empty)	40		false
Sys ID	Sys ID (GUID)	(empty)	32		false
QR Code	String	(empty)	255		false
Source Station	Reference	Metro Station's Details	32		false
Created by	String	(empty)	40		false
Updated	Date/Time	(empty)	40		false
Ticket Number	String	(empty)	50		false
Fare Amount	Integer	(empty)	40	15	true
Created	Date/Time	(empty)	40		false
Insert a new row...					

Subtask 3: Define Field Properties

After creating the fields, several field properties were configured to ensure proper data validation and system functionality.

These properties control how the fields behave within the system.

Field Property Configuration

Some important configurations include:

Mandatory Fields
Important fields such as Source Station, Destination Station, and Travel Date were marked as mandatory to ensure users provide required information.

Data Types
Correct data types were assigned to fields to maintain data consistency. For example:

Date field for Travel Date

Integer field for Passenger Count

String field for Ticket Number

Reference Fields
Source Station and Destination Station were configured as reference fields linked to the Metro Station Details table, enabling users to select stations from a predefined list.

Display Settings
Certain fields were configured to appear in list views and forms for better visibility and system usability.

These configurations help maintain data accuracy and improve system performance.

Subtask 4: Add the Field to a Form (Optional)

Once the table and fields were created, they were added to the Metro Ticket form layout to allow administrators to view and manage ticket records easily.

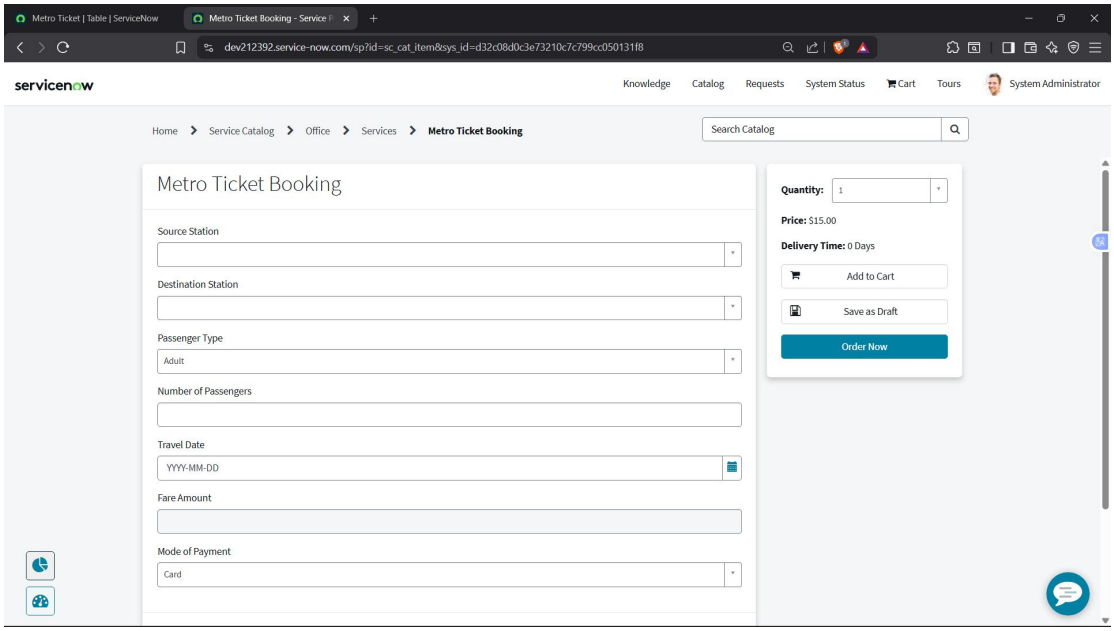
The form layout was customized to display the most important ticket details such as:

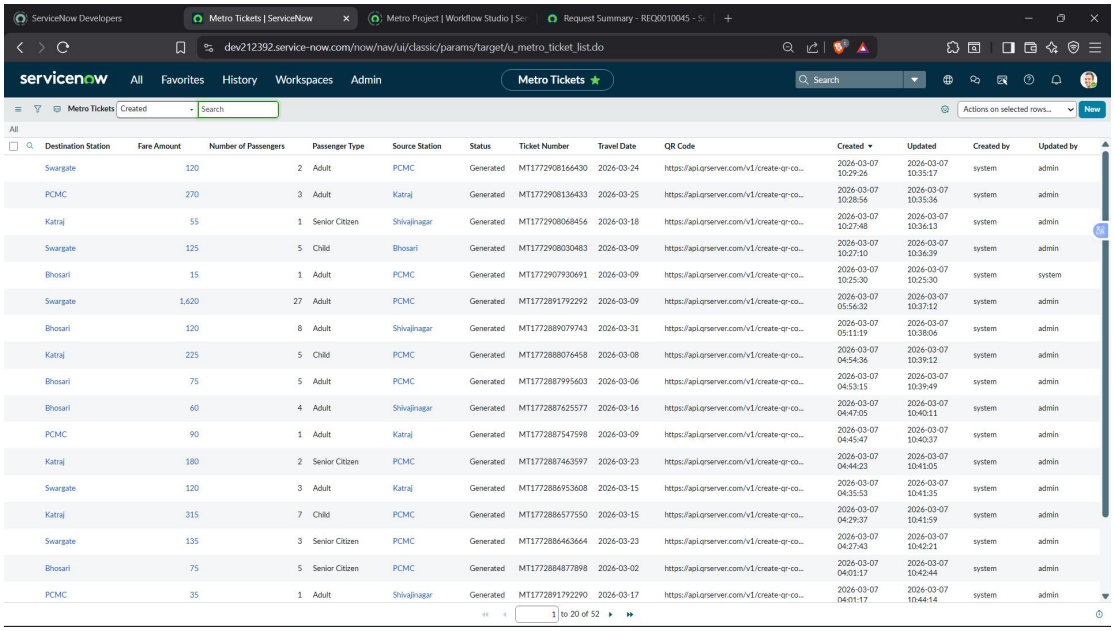
Ticket Number

Source Station
Destination Station
Passenger Type
Number of Passengers
Travel Date
Fare Amount
Ticket Status
QR Code

This allows administrators to open a ticket record and view all relevant travel information in a structured format.

Additionally, list layouts were configured to show important fields in the Metro Ticket table list view, enabling easier monitoring of ticket records.





Automation Logic

Automation logic is responsible for automating the ticket booking workflow in the Metro Ticket Generating System. In this project, automation was implemented using ServiceNow Catalog Client Scripts and Flow Designer to reduce manual intervention and ensure that ticket generation happens automatically after a user submits a booking request.

The automation logic handles several operations such as fare calculation, ticket record creation, ticket number generation, QR code generation, and sending email notifications to users.

By implementing automation in the system, the ticket booking process becomes faster, more efficient, and less prone to human errors.

Subtask 1: Creation & Implementation of Catalog Client Script

A Catalog Client Script was created in the Service Catalog item Metro Ticket Booking to automate fare calculation and validate user inputs dynamically on the form.

This client-side script runs in the user's browser whenever certain form fields change, allowing the system to automatically update values without waiting for the form to be submitted.

Purpose of the Client Script

The catalog client script was implemented to:

Automatically calculate the fare based on the selected Source Station and Destination Station
Multiply the fare based on the Number of Passengers
Automatically populate the Fare Amount field
Provide real-time feedback to users while filling the form
Improve the user experience by reducing manual calculations

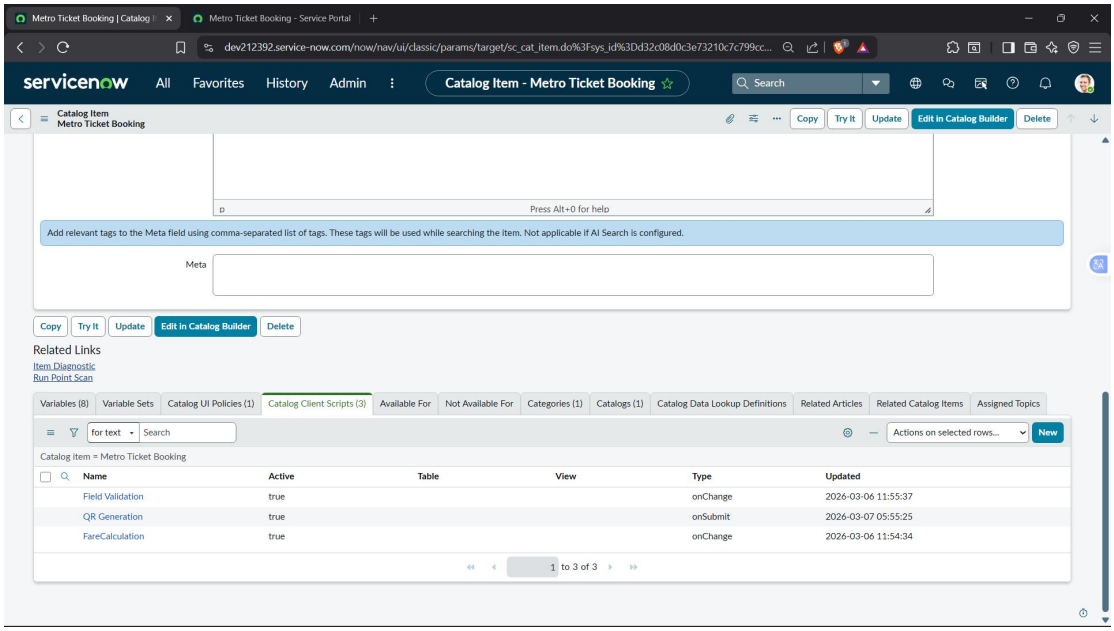
Script Execution

The client script triggers when the user changes important fields such as:

Source Station
Destination Station
Passenger Type
Number of Passengers

When these values change, the script recalculates the fare and updates the Fare Amount field automatically.

This dynamic behavior ensures that users always see the correct ticket price before submitting the booking request.



Subtask 2: Creation & Implementation of Flow Designer Automation

ServiceNow Flow Designer was used to automate the backend processing of ticket bookings. When a user submits the ticket booking form through the Service Catalog, the flow automatically performs several actions to generate and process the ticket.

Flow Trigger

The flow is triggered when the Metro Ticket Booking catalog item is requested.

Trigger Type:
Service Catalog → Catalog Item Requested

This ensures that the automation starts immediately after the user submits the ticket request.

Flow Automation Steps

The flow performs the following automated steps:

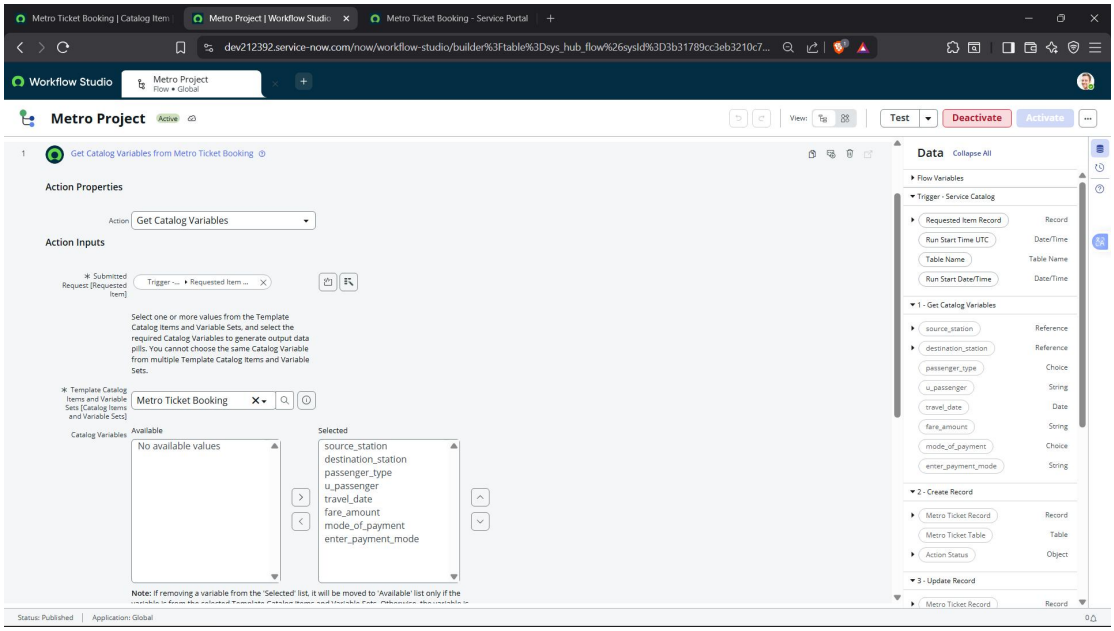
1. Retrieve Catalog Variables

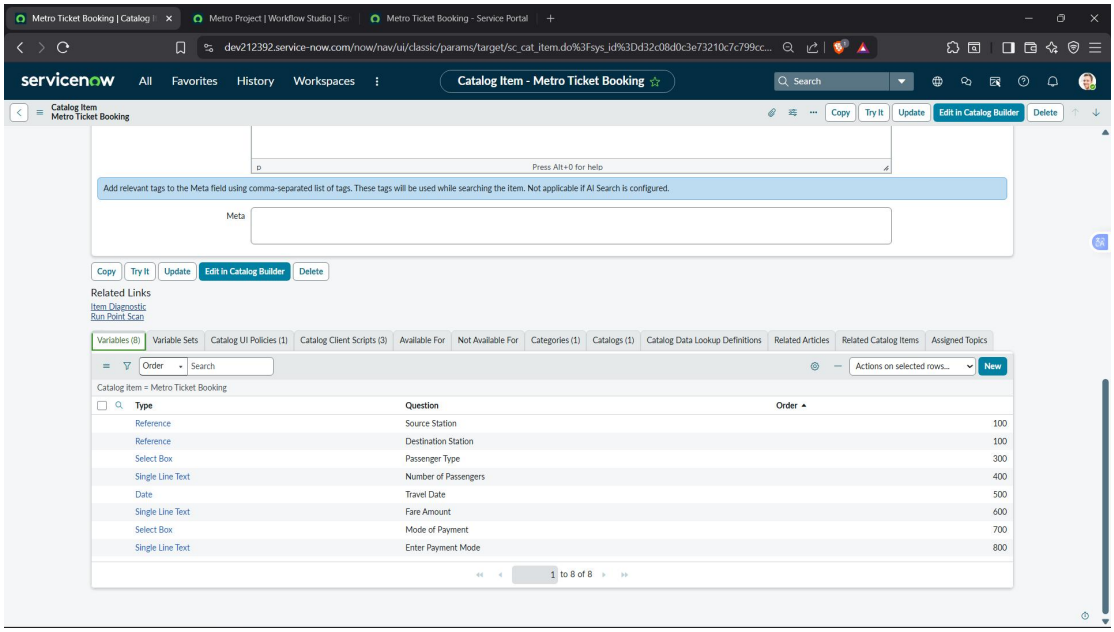
The first step retrieves the values entered by the user in the catalog form.

Examples of retrieved variables include:

Source Station
Destination Station
Passenger Type
Number of Passengers
Travel Date
Mode of Payment

These values are used in later steps to generate the ticket record.





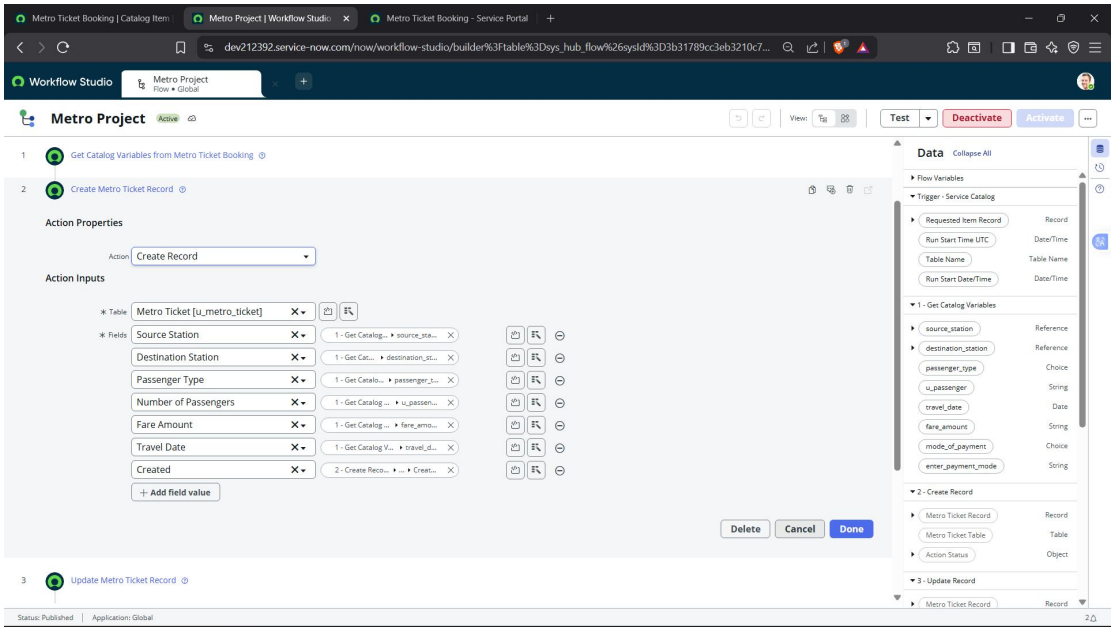
2. Create Metro Ticket Record

A new record is automatically created in the Metro Ticket (u_metro_ticket) table.

The record stores the following information:

Source Station
Destination Station
Passenger Type
Number of Passengers
Travel Date
Fare Amount

This step ensures that every ticket booking creates a corresponding database record.



3. Generate Ticket Number

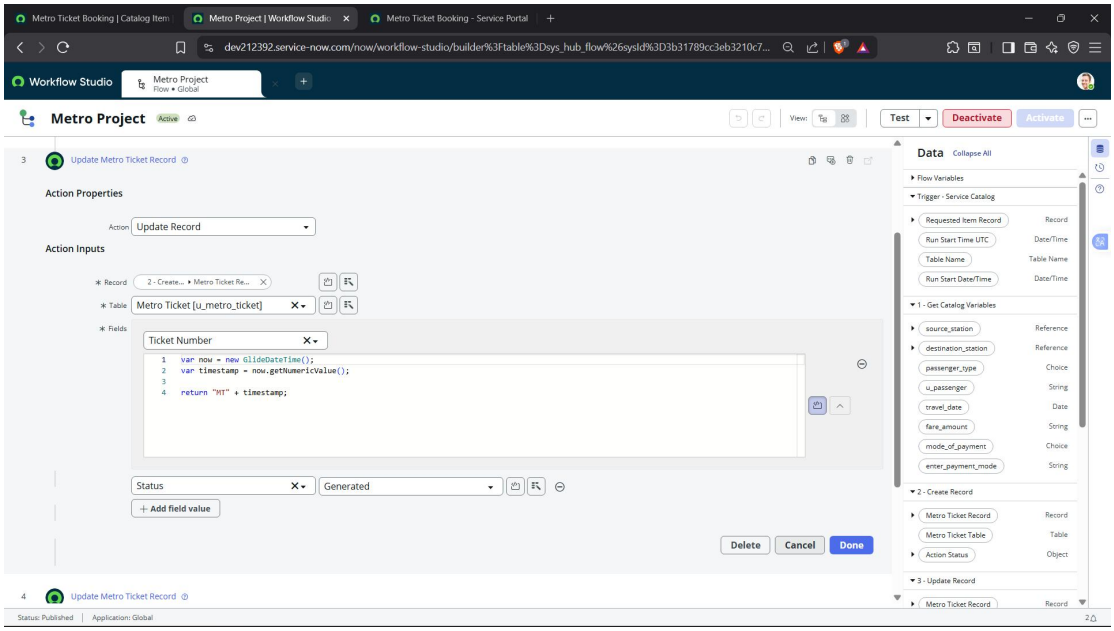
A unique Ticket Number is generated automatically for each booking using a script.

The ticket number follows a format such as:

MT + Timestamp

Example:
MT17288497823

Using timestamps ensures that every ticket number is unique and avoids duplication.



4. Generate QR Code

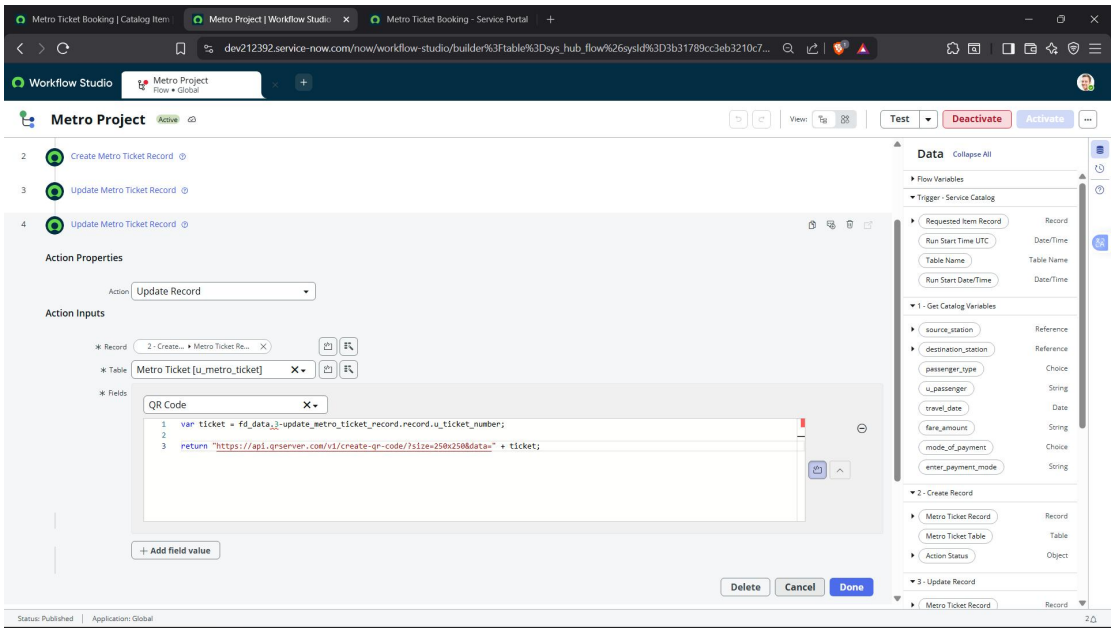
The system generates a QR code for each ticket using an external QR code API.

API Used:

https://api.qrserver.com/v1/create-qr-code/

The QR code contains the ticket number and is stored in the QR Code field of the Metro Ticket record.

Passengers can scan this QR code at the metro station for ticket validation.



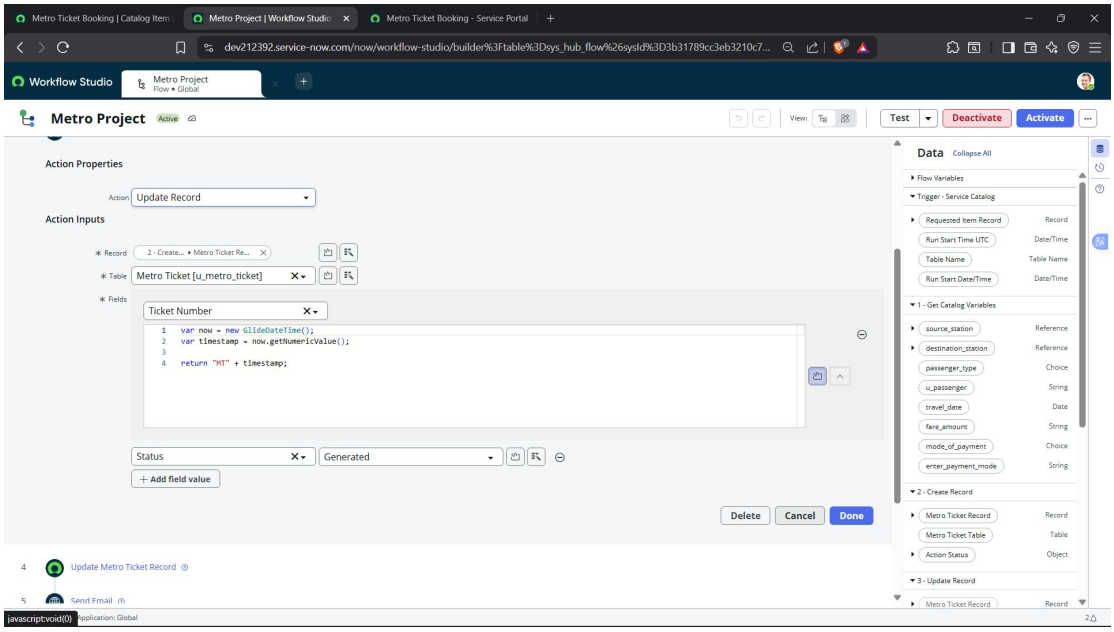
5. Update Ticket Status

After generating the ticket number and QR code, the ticket status is updated automatically.

Example Status Values:

Generated
Pending
Approved

This helps track the current state of the ticket in the system.



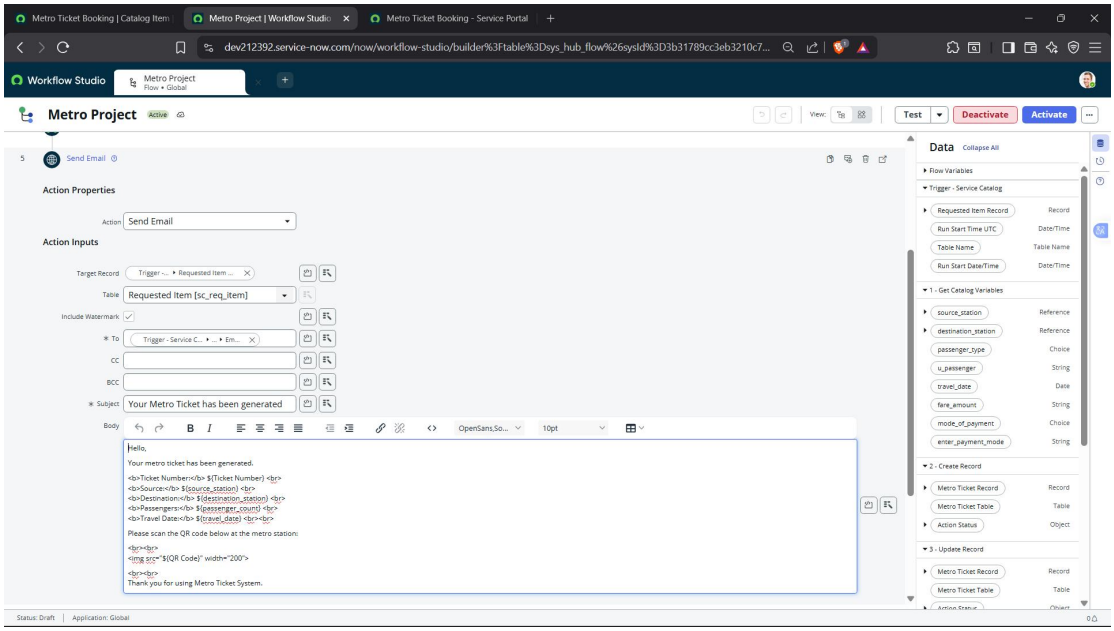
6. Send Email Notification

An automated email is sent to the user after the ticket is generated.

The email contains:

Ticket Number
Source Station
Destination Station
Number of Passengers
Travel Date
QR Code information

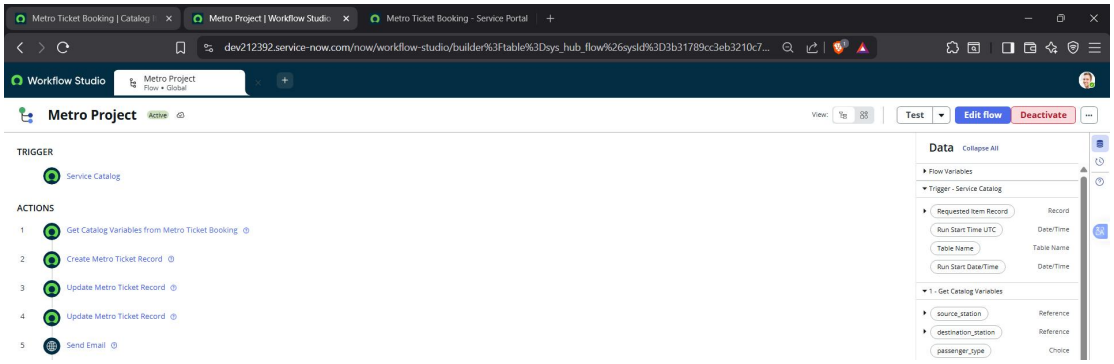
This ensures that passengers receive confirmation of their ticket booking.



Summary of Automation Workflow

The complete automation workflow works as follows:

User submits ticket booking form
↓
Flow Designer Triggered
↓
Retrieve Catalog Variables
↓
Create Metro Ticket Record
↓
Generate Ticket Number
↓
Generate QR Code
↓
Update Ticket Status
↓
Send Email Notification



This automation significantly improves efficiency by eliminating manual ticket generation and ensuring a smooth digital ticket booking experience.

Business Rules

Business rules define the logic, conditions, and constraints that control how the system behaves when users interact with it. In the Metro Ticket Generating System, business logic was implemented using UI Policies and Catalog Client Scripts to control form behavior, automate calculations, and validate user inputs.

These configurations ensure that the ticket booking form behaves dynamically and prevents invalid or incomplete data from being submitted.

The business rules implemented in this project help improve data accuracy, enhance user experience, and enforce system constraints during ticket booking.

Subtask 1: Catalog UI Policy Configuration

A Catalog UI Policy was created to control the visibility and behavior of certain fields in the Metro Ticket Booking form.

UI policies allow administrators to dynamically change form fields based on user selections without writing complex scripts.

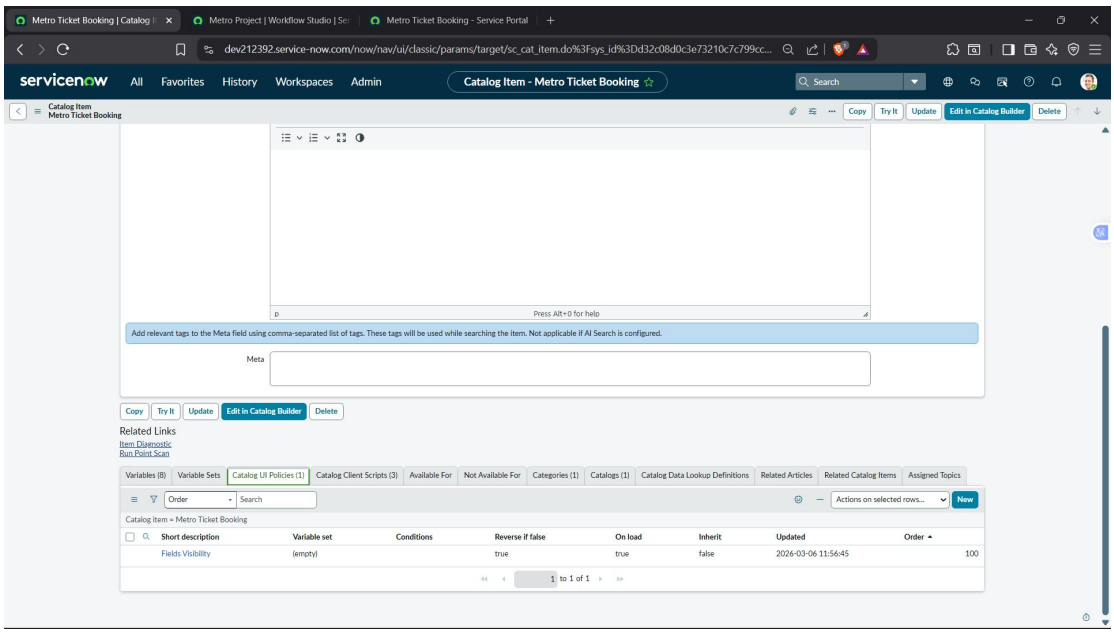
Purpose of the UI Policy

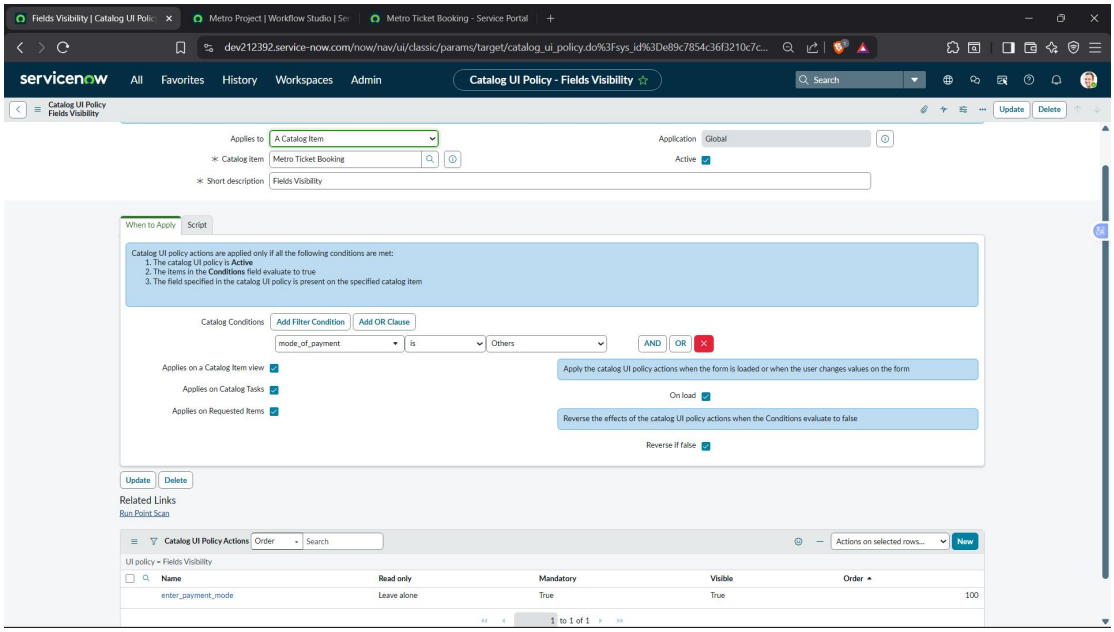
The UI policy was implemented to:
Control field visibility dynamically
Ensure relevant fields appear only when needed
Improve form usability for users
Reduce unnecessary fields on the form

Example Implementation

For example, when certain passenger types or travel options are selected, specific fields can be shown, hidden, or made mandatory depending on the conditions defined in the UI policy.

This ensures that the form remains clean and user-friendly while still collecting all required information.





Subtask 2: Catalog Client Script Configuration

A Catalog Client Script was created to implement automation and dynamic functionality in the ticket booking form.

Catalog Client Scripts run on the client side (in the user's browser) and allow real-time updates to form fields when users interact with the form.

Purpose of the Client Script

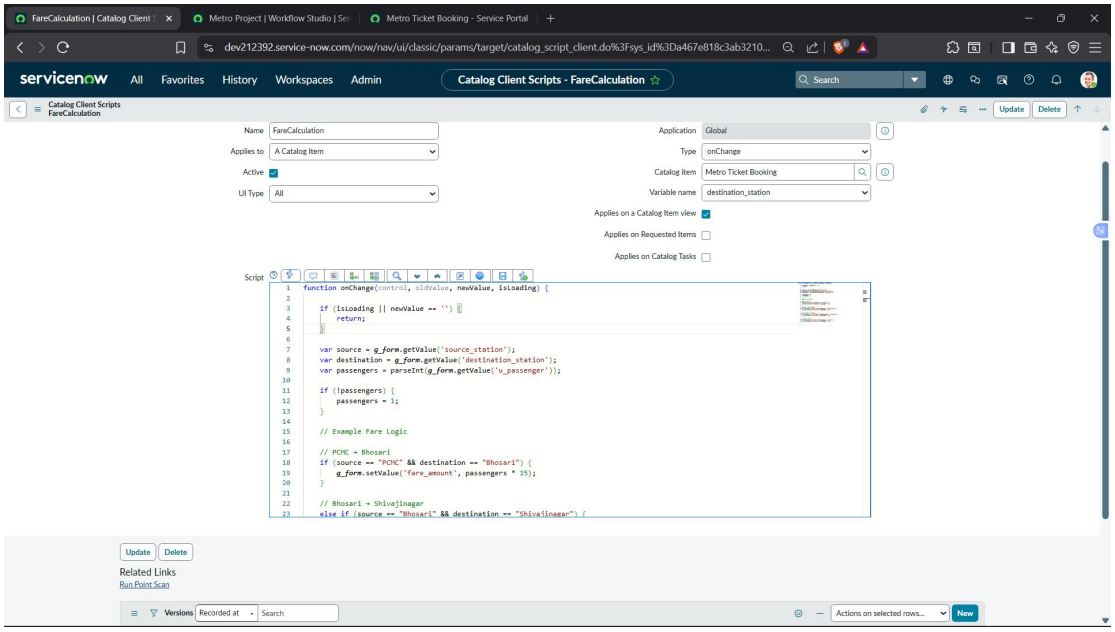
The client script was used to:

Automatically calculate the fare amount
Update the fare when source or destination station changes
Multiply the fare based on the number of passengers
Dynamically update the fare field without reloading the page

How the Script Works

When the user selects the Source Station or Destination Station, the client script calculates the appropriate fare based on predefined fare logic. The calculated value is then automatically displayed in the Fare Amount field.

This ensures that passengers always see the correct ticket price before submitting the request.




```
function onChange(control, oldValue, newValue, isLoading) {

    if (isLoading || newValue == '') {
        return;
    }

    var source = g_form.getValue('source_station');
    var destination = g_form.getValue('destination_station');
    var passengers = parseInt(g_form.getValue('u_passenger'));

    if (!passengers) {
        passengers = 1;
    }

    // Example Fare Logic

    // PCMC → Bhosari
    if (source == "PCMC" && destination == "Bhosari") {
        g_form.setValue('fare_amount', passengers * 15);
    }

    // Bhosari → Shivajinagar
    else if (source == "Bhosari" && destination == "Shivajinagar") {
        g_form.setValue('fare_amount', passengers * 20);
    }

    // Shivajinagar → Swargate
    else if (source == "Shivajinagar" && destination == "Swargate") {
        g_form.setValue('fare_amount', passengers * 25);
    }

    // Swargate → Katraj
    else if (source == "Swargate" && destination == "Katraj") {
        g_form.setValue('fare_amount', passengers * 30);
    }

}
```

Subtask 3: Catalog Client Script for Field Validation

An additional Catalog Client Script was implemented to validate user input fields before the ticket request is submitted.

This validation ensures that incorrect or incomplete data cannot be submitted into the system.

Purpose of the Validation Script

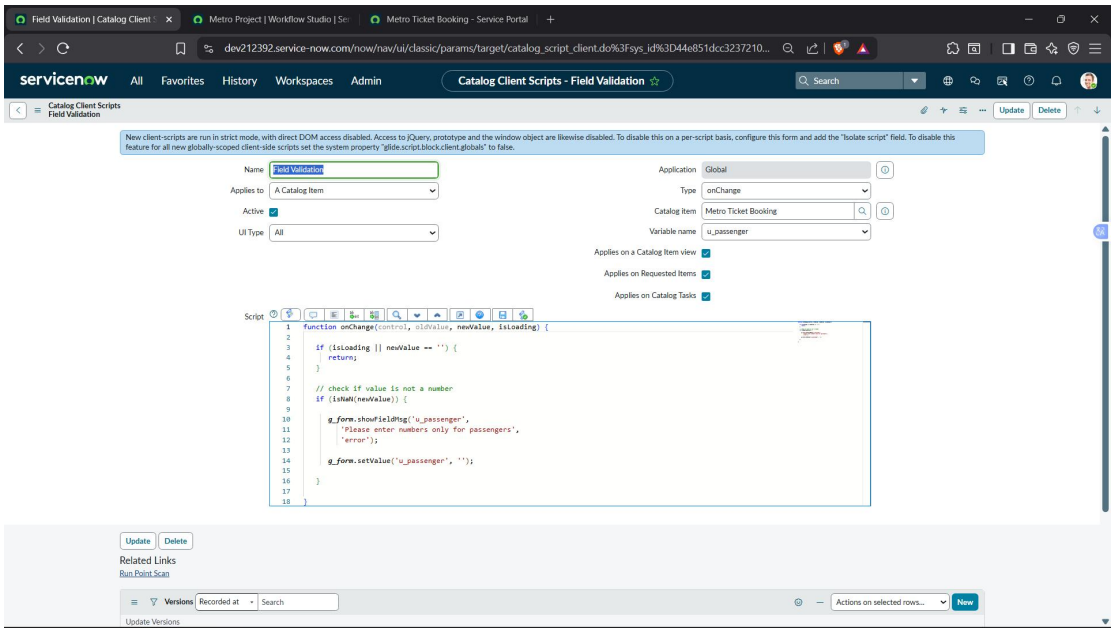
The validation script ensures that:

The Source Station and Destination Station cannot be the same
The Number of Passengers must be greater than zero
Mandatory fields are filled before submission
Invalid data entries are prevented

Benefits of Field Validation

Implementing validation scripts helps to:

Prevent incorrect ticket bookings
Improve data accuracy in the system
Enhance overall system reliability
Provide immediate feedback to users



Summary of Business Rule Implementation

The business logic implemented in the system ensures that:

The ticket booking form behaves dynamically
Fare calculation happens automatically
Invalid data is prevented from entering the system
The user experience remains smooth and intuitive

By combining UI Policies and Client Scripts, the Metro Ticket Generating System maintains data accuracy while simplifying the ticket booking process for users.